Unit 13: Exception Handling

CONTENTS

Objectives

After studying this unit, you will be able to:

- Recognize errors and exception
- Understand Exception Handling
- Explain Caching of multiple exceptions
- Analyze multiple cache statements

Introduction

We know that it is very rare that a program works correctly first time. It might have bugs. The two most common types of bugs are logic errors and syntactic errors. The logic errors occur due to poor understanding of the problem and solution procedure. The syntactic errors arise due to poor understanding of the language itself. We can detect these errors by using exhaustive debugging and testing procedures.

We often come across some peculiar exceptions problems other than logic or syntax errors. They are known as exceptions. Exceptions are run time anomalies or unusual conditions that a program may encounter while executing. Anomalies might include conditions such as division by zero, access to an array outside of its bounds, or running out of memory or disk space. When a program encounters an exceptional condition, it is important that it is identified and dealt with effectively. ANSI C++ provides built-in language features to detect and handle exceptions which are basically run time errors.

Exception handling was not part of the original C++. It is a new feature added to ANSI C++. Today, almost all compilers support this feature. C++ exception handling provides a type-safe, integrated approach, for coping with the unusual predictable problems that arise while executing a program.


Did you know?

Error: Error is an illegal operation performed by the user which results in abnormal working of the program.

13.1 Basics of Exception Handling

Exceptions are of two kinds, namely, synchronous exceptions and asynchronous exceptions. Errors such as "out-of-range index" and "over-flow" belong to the synchronous type exceptions The errors that are caused by events beyond the control of the program (such as keyboard interrupts) are called asynchronous exceptions. The proposed exception handling mechanism in C++ is designed to handle only synchronous exceptions.

The purpose of the exception handling mechanism is to provide means to detect and report an "exceptional circumstances" so that appropriate action can be taken. The mechanism suggests a separate error handling code that performs the following tasks:

1. Find the problem I Hit the exception).

2. Inform that an en or has occurred (Throw the exception).

3. Receive the error information (Catch the exception).

4. Take corrective actions (Handle the exception).

The error handling code basically consists of two segments, one to detect errors and to throw exceptions, and the other to catch the exceptions and to take appropriate actions.

 Did you know?

Compile Time Error: Errors caught during compiled time is called Compile time errors. Compile time errors include library reference, syntax error or incorrect class import.

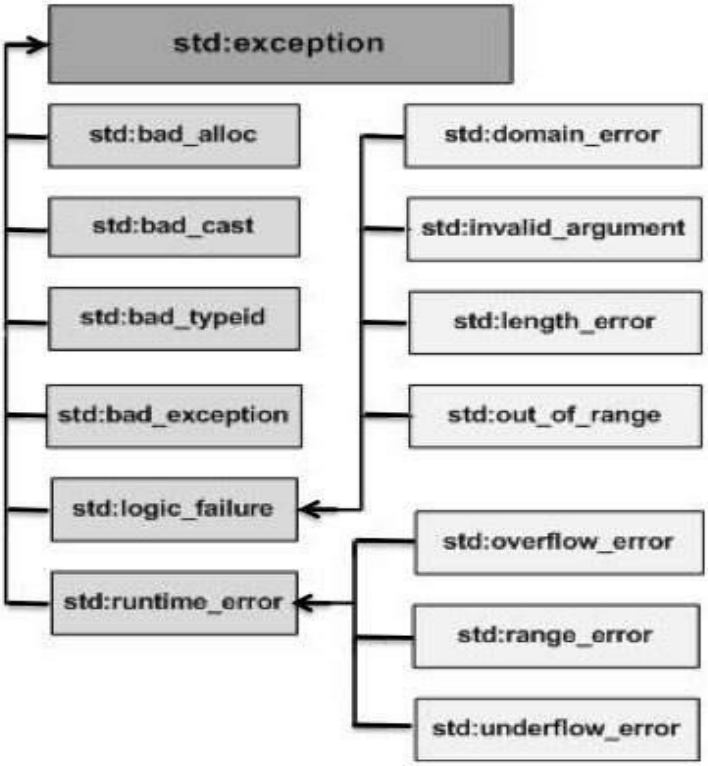
Run Time Error: They are also known as exceptions. An exception caught during run time creates serious issues.

Exceptions are preferred in modern C++ for the following reasons:

- An exception forces calling code to recognize an error condition and handle it. Unhandled exceptions stop program execution.
- An exception jumps to the point in the call stack that can handle the error. Intermediate functions can let the exception propagate. They don't have to coordinate with other layers.
- The exception stack-unwinding mechanism destroys all objects in scope after an exception is thrown, according to well-defined rules.
- An exception enables a clean separation between the code that detects the error and the code that handles the error.

C++ Standard Exceptions

C++ provides a list of standard exceptions defined in <exception> which we can use in our programs. These are arranged in a parent-child class hierarchy shown below –



Description is here

Sr.No	Exception & Description
1	**std::exception** An exception and parent class of all the standard C++ exceptions.
2	**std::bad_alloc** This can be thrown by new.
3	**std::bad_cast** This can be thrown by dynamic_cast.
4	**std::bad_exception** This is useful device to handle unexpected exceptions in a C++ program.
5	std::bad_typeid This can be thrown by typeid.
6	**std::logic_error** An exception that theoretically can be detected by reading the code.
7	**std::domain_error** This is an exception thrown when a mathematically invalid domain is used.
8	**std::invalid_argument** This is thrown due to invalid arguments.

9	**std::length_error** This is thrown when a too big std::string is created.
10	**std::out_of_range** This can be thrown by the 'at' method, for example a std::vector and std::bitset<>::operator[]().
11	**std::runtime_error** An exception that theoretically cannot be detected by reading the code.
12	**std::overflow_error** This is thrown if a mathematical overflow occurs.
13	**std::range_error** This is occurred when you try to store a value which is out of range.
14	**std::underflow_error** This is thrown if a mathematical underflow occurs.

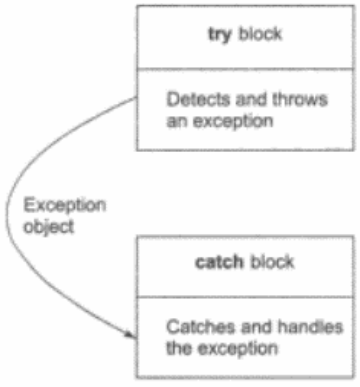
13.2 Exception Handling Mechanism

Exceptions are run time anomalies or unusual conditions that a program may encounter while executing. Anomalies might include conditions such as division by zero, access to an array outside of its bounds, or running out of memory or disk space. When a program encounters an exceptional condition, it is important that it is identified and dealt with effectively.

The mechanism performs following tasks:

- Find the problem (Hit the exception).
- Inform that an error has occurred (Throw the exception).
- Receive the error information (Catch the expression).
- Take corrective actions (Handle the exceptions).
- The error handling code basically consists of two segments one to detect errors and to throw exceptions, and other to catch the exceptions and to take appropriate actions.

C++ exception handling mechanism is basically built upon three keywords, namely, try, throw, and catch. The keyword try is used to preface a block of statements (surrounded by braces) which may generate exceptions. This block of statements is known as try block. When an exception is detected, it is thrown using a throw statement in the try block. A catch block defined by the keyword catch 'catches' the exception 'thrown' by the throw statement in the try block, and handles it appropriately. The relationship is shown in Figure.



The catch block that catches an exception must immediately follow the try block that throws the exception. The general form of these two blocks are as follows:

```
…….
……..
try
{
…….
throw exception;            // Block of statements which
…….                         // detects and throws on exception
……..
}
catch(type org)  {                  // Catches exception
…….                                 // Block of statements that
……..                                       // handles the exception
}
……..
……..
```

When the try block throws an exception, the program control leaves the try block and enters the catch statement of the catch block. Note that exceptions are objects used to transmit information about a problem. If the type of object thrown matches the arg type in the catch statement, then catch block is executed for handling the exception. If they do not match, the program is aborted with the help of the abort() function which is invoked by default. When no exception is detected and thrown, the control goes to the statement immediately after the catch block. That is, the catch block is skipped. This simple try-catch mechanism is illustrated in Program.

```
#include <iostream>
using namespace std;
int main () {
int x;
cout<<"Enter number";
cin>>x;
try {
  if (x >=1) {
    cout << "Input is valid";
  } else {
    throw (x);
  }
}
catch (int num) {
  cout << "Input Invalid\n";
  cout << "Number is: " << num;
}
}
```

Output

```
Enter number12
Input is valid
Process returned 0 (0x0)   execution time : 1.927 s
Press any key to continue.
```

Note

- Exception Handling in C++ is a process to handle runtime errors.
- A C++ exception is a response to an exceptional circumstance that arises while a program is running, such as an attempt to divide by zero.

Did you know?

What is an exception?

Exceptions are run time anomalies or unusual conditions that a program may encounter while executing.

Exception Handling Keywords

- **throw-** when a program encounters a problem, it throws an exception. The throw keyword helps the program perform the throw.
- **catch-** a program uses an exception handler to catch an exception. It is added to the section of a program where you need to handle the problem. It's done using the catch keyword.
- **try-** the try block identifies the code block for which certain exceptions will be activated. It should be followed by one/more catch blocks.

Advantages of Exception Handling

1. **Separating Error-Handling Code from "Regular" Code: -** Exceptions provide the means to separate the details of what to do when something out of the ordinary happens from the main logic of a program. In traditional programming, error detection, reporting, and handling often lead to confusing spaghetti code. Exceptions enable you to write the main flow of your code and to deal with the exceptional cases elsewhere.

2. **Propagating Errors Up the Call Stack: -** A second advantage of exceptions is the ability to propagate error reporting up the call stack of methods.

3. **Grouping and Differentiating Error Types: -** Because all exceptions thrown within a program are objects, the grouping or categorizing of exceptions is a natural outcome of the class hierarchy.

Task: Write a program to demonstrate working of try, throw and catch blocks in C++.

Multiple Cache Statements

It is possible that a program segment has more than one condition to throw an exception. In such cases, we can associate more than one catch statement with a try as shown below:

```
try

{           // try block
……..
}

catch(type1 arg)

{

// catch block1

} catch(type2 arg) {

// catch block2

}
 ……..

……

catch(typeN arg) {

// catch blockN
}
```


Note

- Each catch block catches one type of exceptions
- Need multiple catch blocks
- When the value is not important, the parameter can be omitted.
    - E.g. catch(int) {…}
- catch (…) catches any exception, can serve as the default exception handler
- Although a try-block followed by its catch-block can be nested inside another try-block
- Although a try-block followed by its catch-block can be nested inside another try-block

 Did you know?

Why multiply blocks are useful?

Multiple catch blocks are used when we have to catch a specific type of exception out of many possible type of exceptions i.e. an exception of type char or int or short or long etc.

 Lab Exercise

```
//Program
#include<iostream>
using namespace std;
class compareAges{
public:
   int fathersAge;
   int sonsAge;
   void inputAge(){
      cout<<"Enter Fathers Age";
cin>>fathersAge;
cout<<"Enter Sons Age";
cin>>sonsAge;
   }
   void compare(){
   try{
   if(fathersAge==sonsAge) throw 55;
   else if(sonsAge>fathersAge) throw ('x');
   else throw (2.1f);
   }
   catch(int a){
   cout<<"Invalid..";
   }
   catch(char b){
   cout<<"Age of son is never greater than the age of father";
   }
   catch (float c){
   cout<<"Valid input";
   }
   }
};
main(){
compareAges obj;
obj.inputAge();
obj.compare();
}
```

Output

```
Enter Fathers Age95
Enter Sons Age100
Age of son is never greater than the age of father
Process returned 0 (0x0)   execution time : 16.626 s
Press any key to continue.
```

13.3 Caching Multiple Exceptions

Multiple catch blocks are used when we have to catch a specific type of exception out of many possible type of exceptions i.e. an exception of type char or int or short or long etc. Let's see the use of multiple catch blocks with an example.

Lab Exercise

```
#include<iostream>


using namespace std;


int main()
{
int a=10, b=0, c;
try
{
        //if a is divided by b(which has a value 0);
        if(b==0)
                throw(c);
        else
        c=a/b;


}
catch(char c)                                   //catch block to handle/catch exception
{
        cout<<"Caught exception : char type ";
}
catch(int i)                                     //catch block to handle/catch exception
{
        cout<<"Caught exception : int type ";
}
catch(short s)                          //catch block to handle/catch exception
{
        cout<<"Caught exception : short type ";
}
cout<<"\n Hello World";
```

}

Output

```
Caught exception : int type
 Hello World
Process returned 0 (0x0)   execution time : 0.845 s
Press any key to continue.
```

Summary

- A try block may throw an exception directly or invoke a function that throws an exception. Irrespective of location of the throw point, the catch block is placed immediately after the try block.
- We can place two or more catch blocks together to catch and handle multiple types of exceptions thrown by a try block.
- It is also possible to make a catch statement to catch all types of exceptions using ellipses as its argument.
- We may also restrict a function to throw only a set of specified exceptions by adding a throw specification clause to the function definition.

Keywords

Error - Error is an illegal operation performed by the user which results in abnormal working of the program.

Exception – An exception is a problem that arises during the execution of a program. A C++ exception is a response to an exceptional circumstance that arises while a program is running, such as an attempt to divide by zero.

Run time error - They are also known as exceptions. An exception caught during run time creates serious issues.

Throw- when a program encounters a problem, it throws an exception. The throw keyword helps the program perform the throw.

Catch- a program uses an exception handler to catch an exception. It is added to the section of a program where you need to handle the problem. It's done using the catch keyword.

Try- the try block identifies the code block for which certain exceptions will be activated. It should be followed by one/more catch blocks.

Self Assessment

1. Which is used to handle the exceptions in c++?

A. catch handler
B. handler
C. exception handler
D. throw

2. Which type of program is recommended to include in try block?

A. static memory allocation
B. dynamic memory allocation
C. const reference
D. pointer

3. Which statement is used to catch all types of exceptions?
A. catch()
B. catch(Test t)
C. catch(…)
D. catch(Test)

4. What is an exception in C++ program?
A. A problem that arises during the execution of a program
B. A problem that arises during compilation
C. Also known as the syntax error
D. Also known as semantic error

5. By default, what a program does when it detects an exception?
A. Continue running
B. Results in the termination of the program
C. Calls other functions of the program
D. Removes the exception and tells the programmer about an exception

6. Why do we need to handle exceptions?
A. To avoid unexpected behaviour of a program during run-time
B. To let compiler remove all exceptions by itself
C. To successfully compile the program
D. To get correct output

7. How Exception handling is implemented in the C++ program?
A. Using Exception keyword
B. Using try-catch block
C. Using Exception block
D. Using Error handling schedules

8. Which is used to throw a exception?
A. Try
B. Throw
C. Catch
D. None of the above

9. How do define the user-defined exceptions?
A. Inherting & overriding exception class functionlity
B. Overriding class functionlity
C. Inherting class functionlity
D. None of the above

10. What is an error in C++?
A. Violation of syntactic and semantic rules of a languages
B. Missing of Semicolon
C. Missing of double quotes
D. Violation of program interface

11. We can prevent a function from throwing any exceptions.
A. TRUE
B. FALSE
C. May Be
D. Can't Say

12. An exception can be of only built-In type.
A. True
B. False

13. Generic catch handler must be placed at the end of all the catch handlers.
A. True
B. False

14. A try block can be nested under another try block.
A. True
B. False

15. What is the difference between error and exception?
A. Both are the same
B. Errors can be handled at the run-time but the exceptions cannot
C. Exceptions can be handled at the run-time but the errors cannot
D. Both can be handled during run-time

Answers for Self Assessment

1.	C	2.	B	3.	C	4.	A	5.	B
6.	A	7.	B	8.	B	9.	A	10.	A
11.	A	12.	B	13.	A	14.	A	15.	C

Review Questions

1. What do you mean by an error?
2. What is an exception? How it is different from error.
3. What is an exception?
4. How is an exception handle in C++?
5. List the advantages of exception handling.
6. What are the basic implementations of exception handling?
7. Explain in detail "exception handling mechanism".
8. When do we use multiple cache statements?
9. What are main keywords that responsible for exception handling in C++.
10. What should we place inside the catch block?

Further Readings

E Balagurusamy; Object-oriented Programming with C++; Tata Mc Graw-Hill.

Herbert Schildt; The Complete Reference C++; Tata Mc Graw Hill.

Robert Lafore; Object-oriented Programming in Turbo C++; Galgotia.

Web Links

https://study.com/academy/lesson/practical-application-for-c-plus-plus-programming-

https://www.codecademy.com/learn/learn-c-plus-plus

https://www.guru99.com/cpp-file-read-write-open.html